CSE 3075

Mobile Application Development

Lab Manual

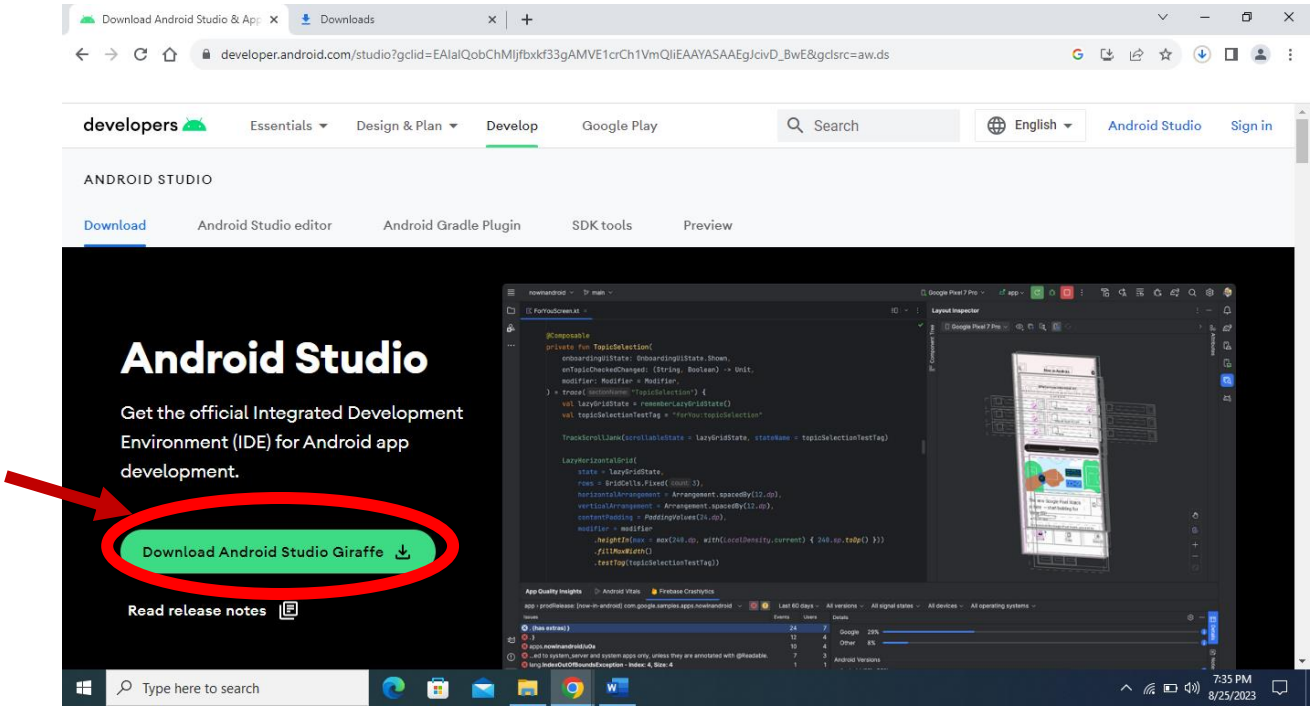
A.Y 2023-24 Odd Sem

Introduction to Android Studio

Download and Install Instructions:

Step 1: Download android studio giraffe (latest version) using the below link:

https://developer.android.com/studio?gclid=EAIaIQobChMIjfbxkf33gAMVE1crCh1VmQIiEAAYASAAEgJcivD_BwE&gclsrc=aw.ds



Download the executable file by accepting the terms and conditions.

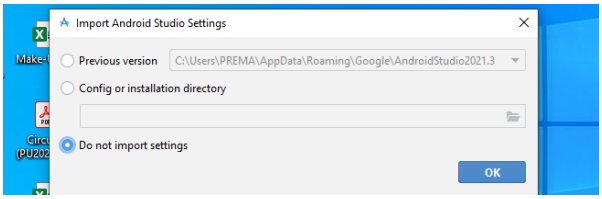
Step 2: Double Click on executable file

Continue by clicking next button until finished installation

Step 3: Import Android Studio Settings

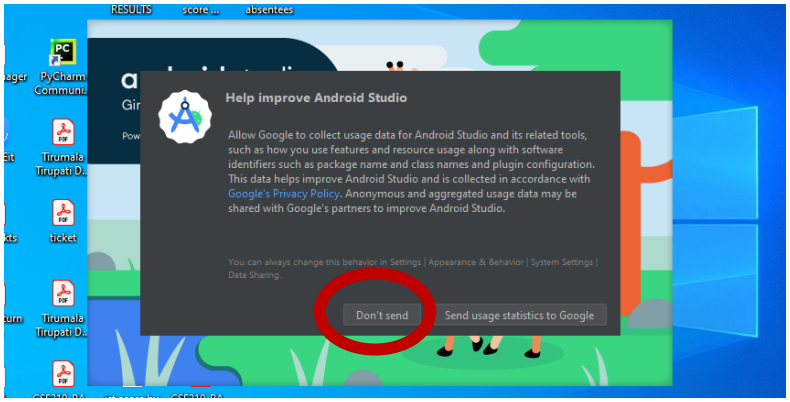
In this window, it will show two different options of config or installation directory and do not import settings

Select Do not import settings, means all your android projects will stored in C drive

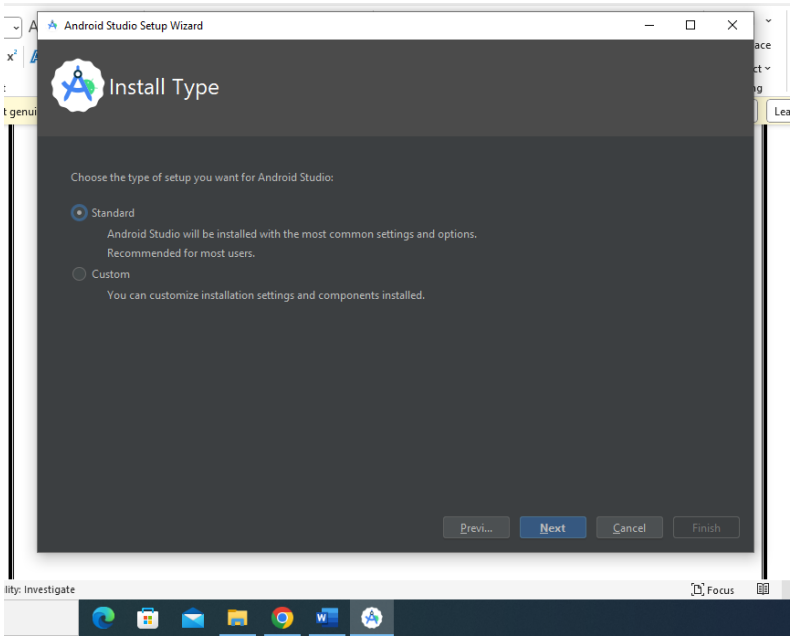


Step 4: Usage Statistics

Don't Send the statistics to google

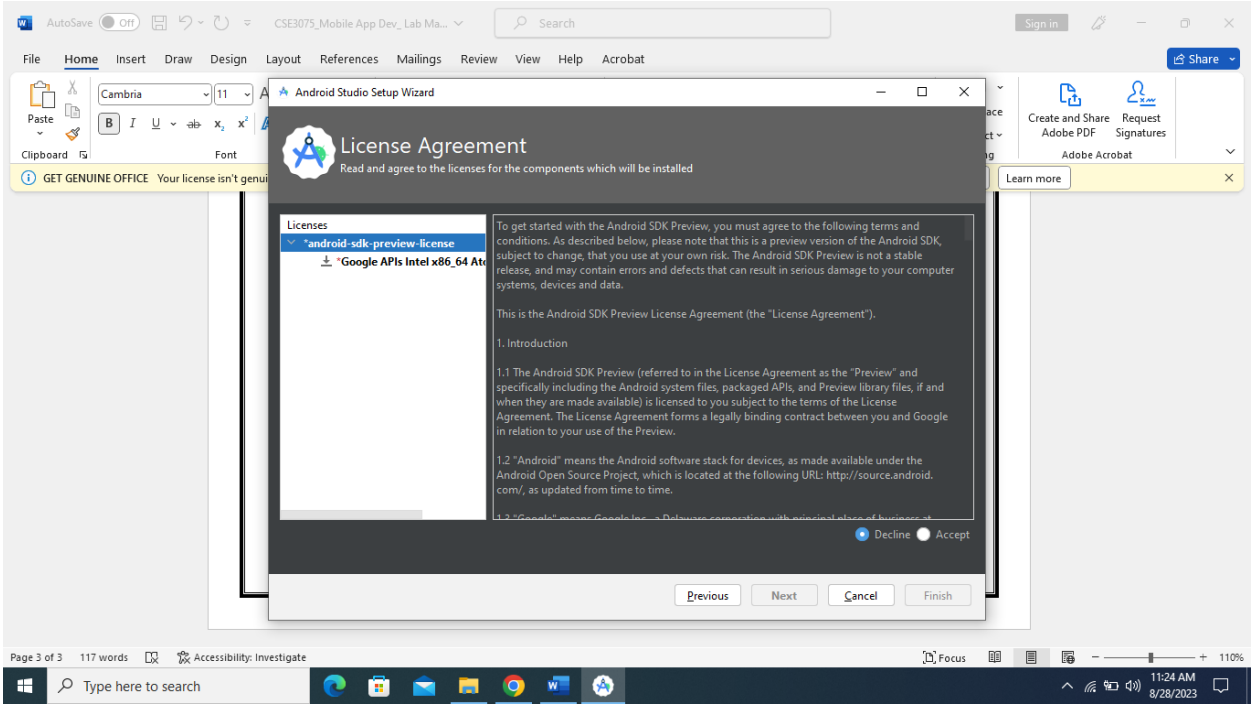


Step 5: Use Standard type of android studio



Step 6: Continue to click next button till license agreement

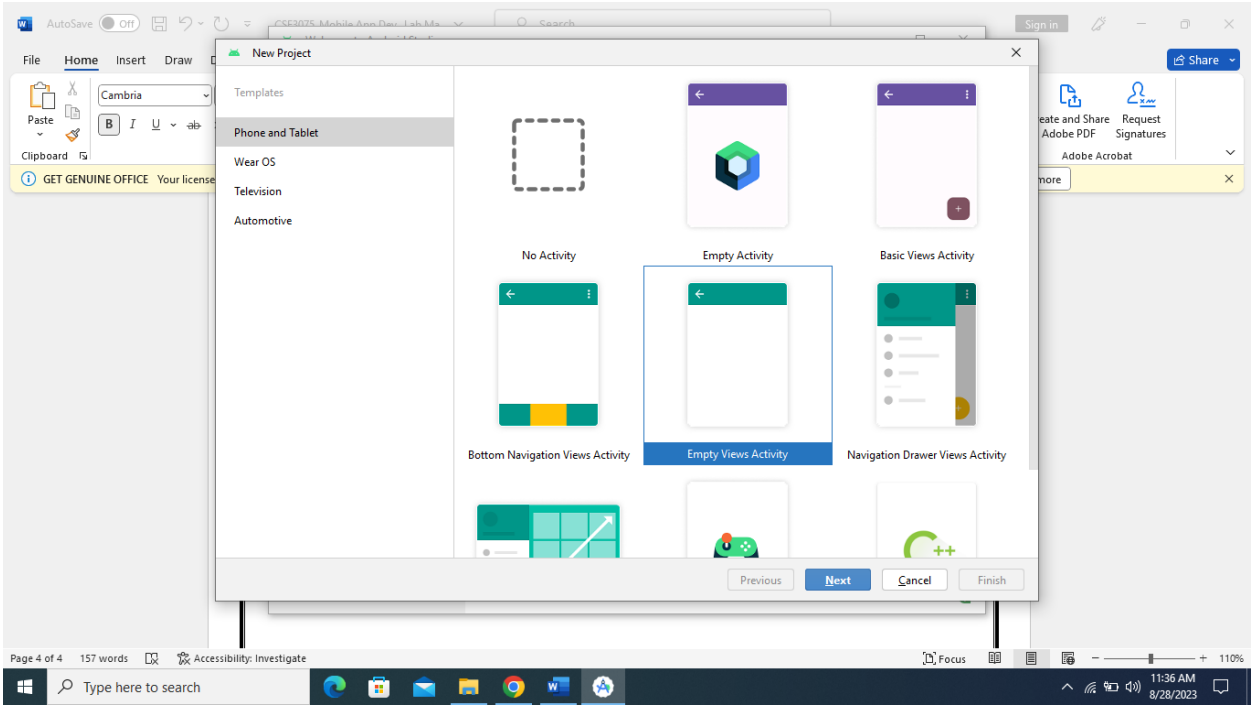
In this window there will be three licenses. Accept all licenses and click finish button



About Android Studio:

Step 1: Create a New Project

Step 2: Choose the Empty Views Activity template in Phone and Tablet category

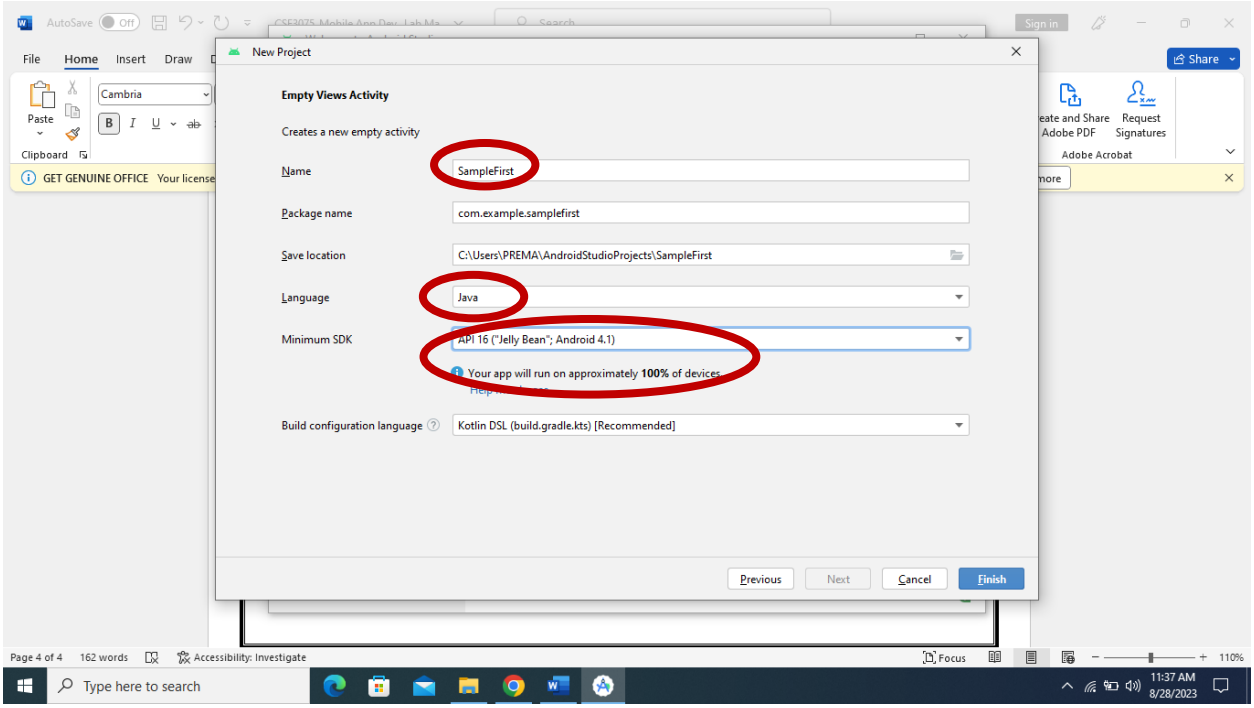


Step 3: Project Name and Language Selection

Give the project name without spaces

Choose the Language Java

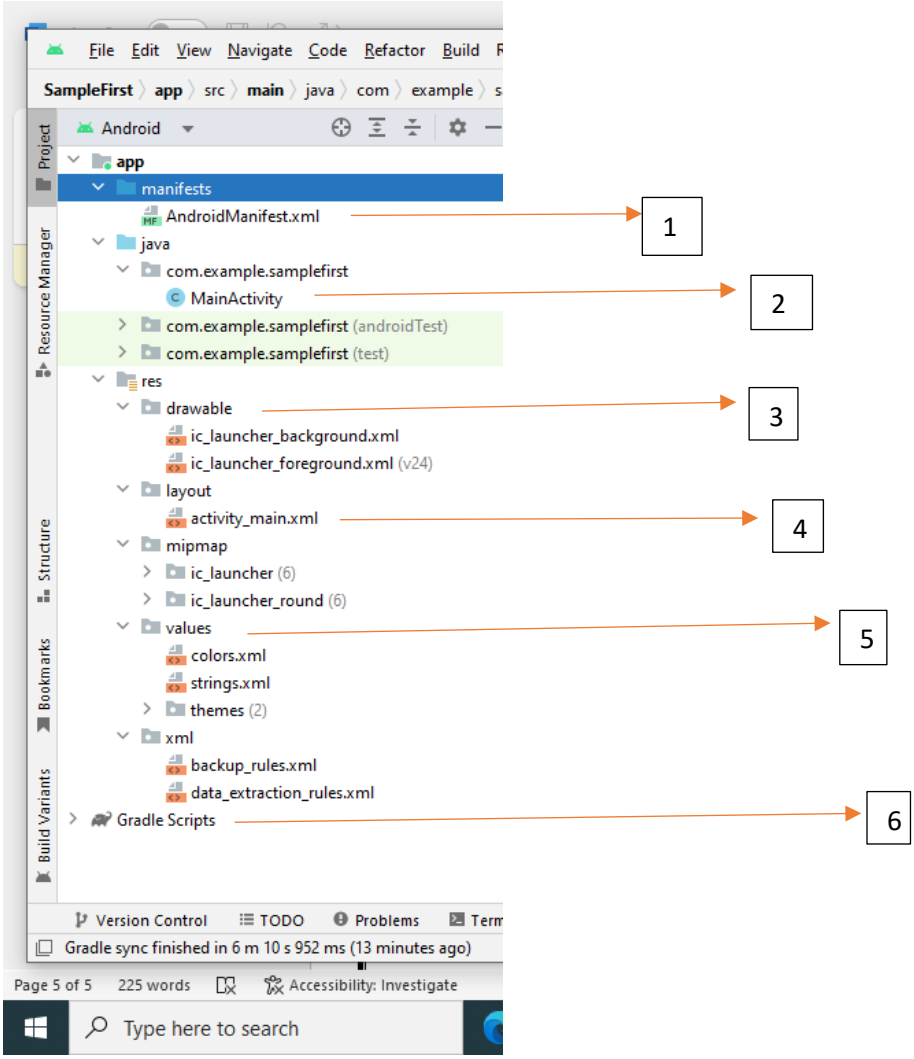
Better to choose Minimum SDK Version, so that your application will run in all devices



Step 4: Wait until Gradle is sync with project

Anatomy of Android Application

Before you run app, you should be aware of a few directories and files in the Android Project:



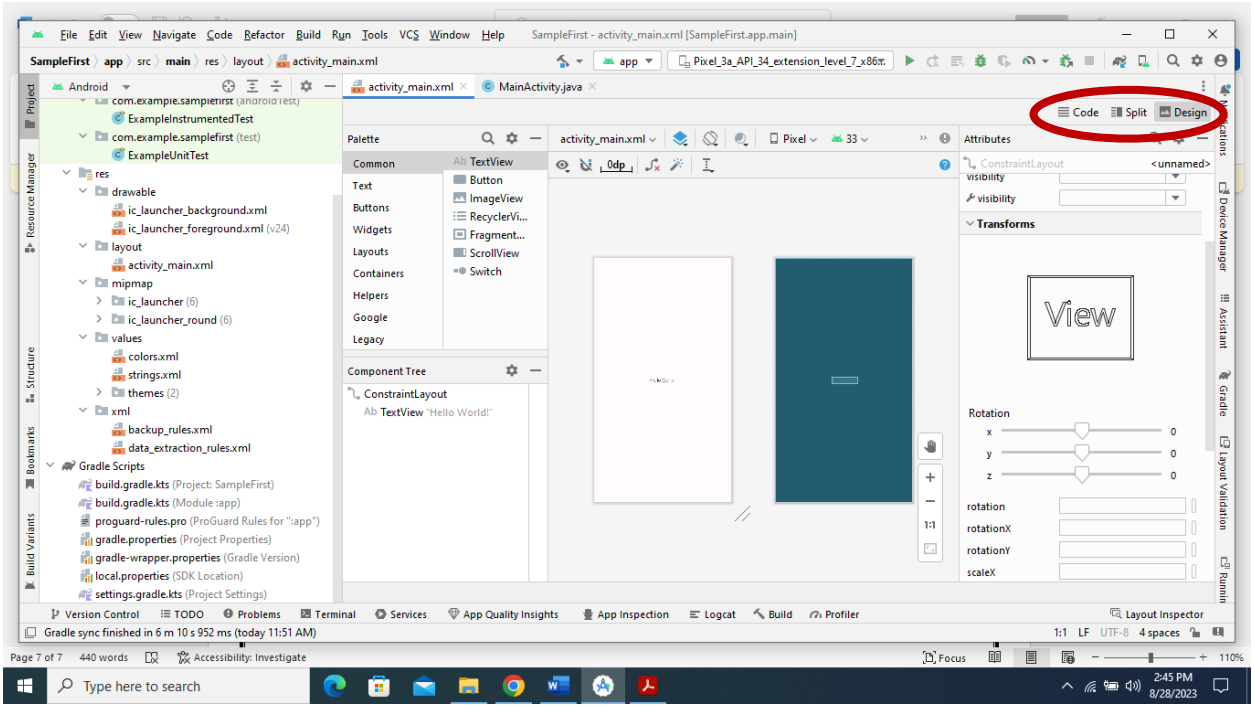
Sl. No	Folder, File & Description
1	Android Manifest.xml This is the manifest file which describes the fundamental characteristics of the app and defines each of its components
2	Java/com.example.samplefirst This contains the .java source files for your project. By default, it includes an MainActivity.java source file having an activity class that runs when your app is launched using the app icon.
3	res/drawable This is a directory for drawable objects that are designed for high- density screens.
4	res/layout This is a directory for files that define your app's user interface.

5	res/values This is a directory for other various XML files that contain a collection of resources, such as strings and colors definitions.
6	Gradle Scripts This directory includes all settings and properties to build your application into virtual emulator

Following section will give a brief overview few of the important application files.

The Layout File:

The activity_main.xml is a layout file available in res/layout directory, that is referenced by your application when building its interface. You will modify this file very frequently to change the layout of your application. You can modify the design either using code tab or design tab. For your "Hello World!" application, this file will have following content related to default layout:




```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity">

   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Hello World!"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
```

```
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

The Main Activity File

The main activity code is a Java file MainActivity.java. This is the actual application file which ultimately gets converted to a Dalvik executable and runs your application. Following is the default code generated by the application wizard for Hello World! application:

```
package com.example.samplefirst;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
  }
}
```

Here, R.layout.activity_main refers to the activity_main.xml file located in the res/layout folder. The onCreate() method is one of many methods that are fixed when an activity is loaded.

The Manifest File

Whatever component you develop as a part of your application, you must declare all its components in a manifest file called AndroidManifest.xml which ressides at the root of the application project directory. This file works as an interface between Android OS and your application, so if you do not declare your component in this file, then it will not be considered by the OS. For example, a default manifest file will look like as following file:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.SampleFirst"
    tools:targetApi="31">
    <activity
      android:name=".MainActivity"
      android:exported="true">
```

```
        <intent-filter>
          <action android:name="android.intent.action.MAIN" />

          <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
    </application>

</manifest>
```

Here <application>...</application> tags enclosed the components related to the application. Attributeandroid:icon will point to the application icon available under res/drawable-hdpi. The application uses the image named ic_launcher.png located in the drawable folders

The <activity> tag is used to specify an activity and android:name attribute specifies the fully qualified class name of the Activity subclass and the android:label attributes specifies a string to use as the label for the activity. You can specify multiple activities using <activity> tags.

The action for the intent filter is named android.intent.action.MAIN to indicate that this activity serves as the entry point for the application. The category for the intent-filter is namedandroid.intent.category.LAUNCHER to indicate that the application can be launched from the device's launcher icon.

The @string refers to the strings.xml file explained below. Hence, @string/app_name refers to theapp_name string defined in the strings.xml fi le, which is "HelloWorld". Similar way, other strings get populated in the application.

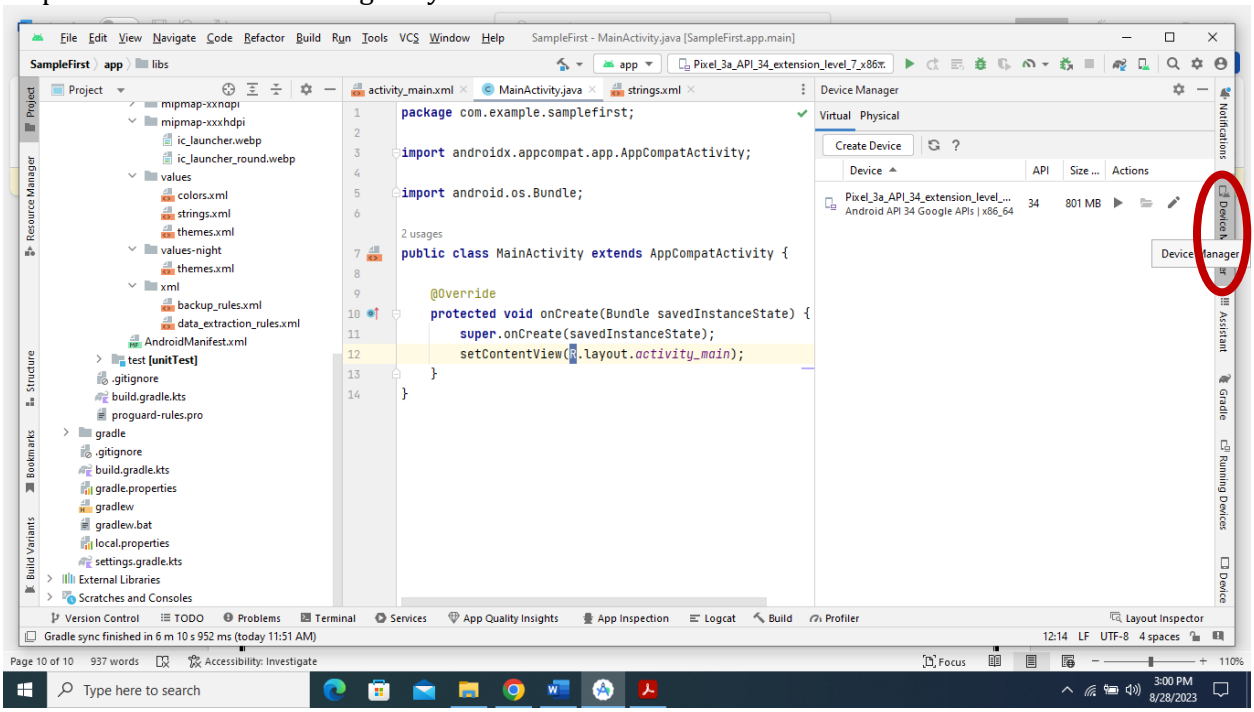
Following is the list of tags which you will use in your manifest file to specify different Android application components:
<activity>elements for activities
<service> elements for services
<receiver> elements for broadcast receivers
<provider> elements for content providers The Strings File

The strings.xml file is located in the res/values folder and it contains all the text that your application uses. For example, the names of buttons, labels, default text, and similar types of strings go into this file. This file is responsible for their textual content. For example, a default strings file will look like as following file:

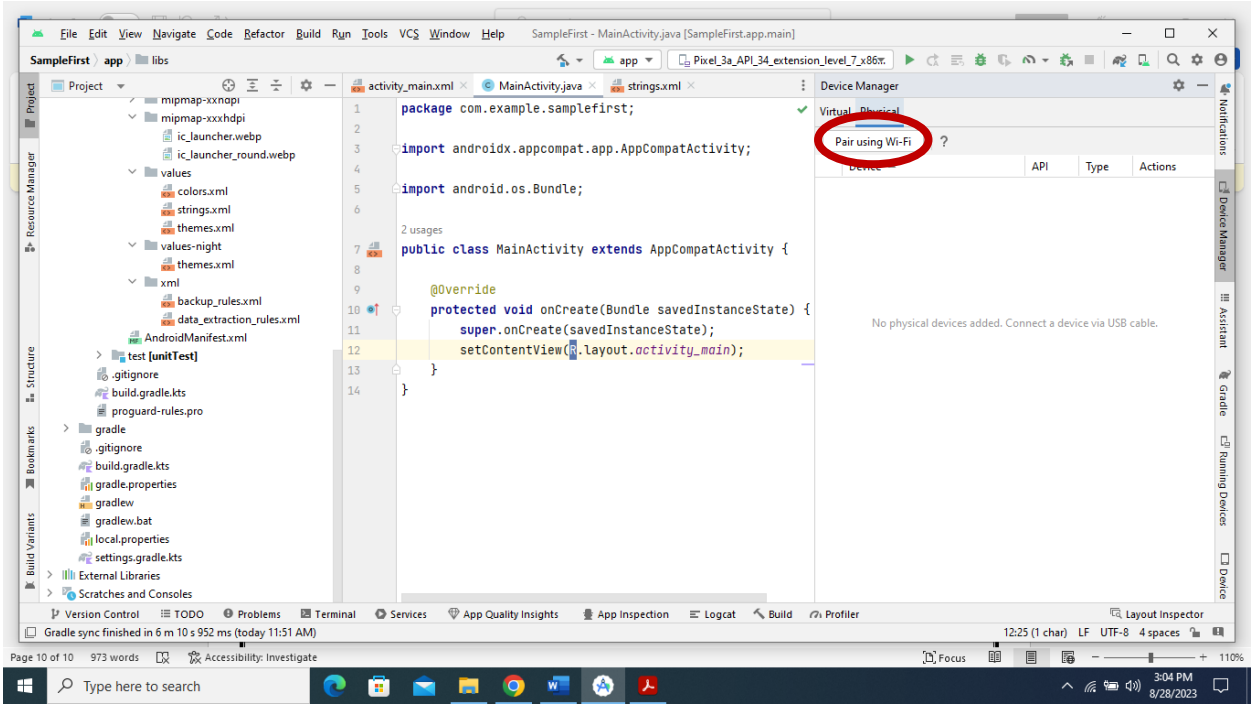
```
<resources>
  <string name="app_name">SampleFirst</string>
</resources>
```

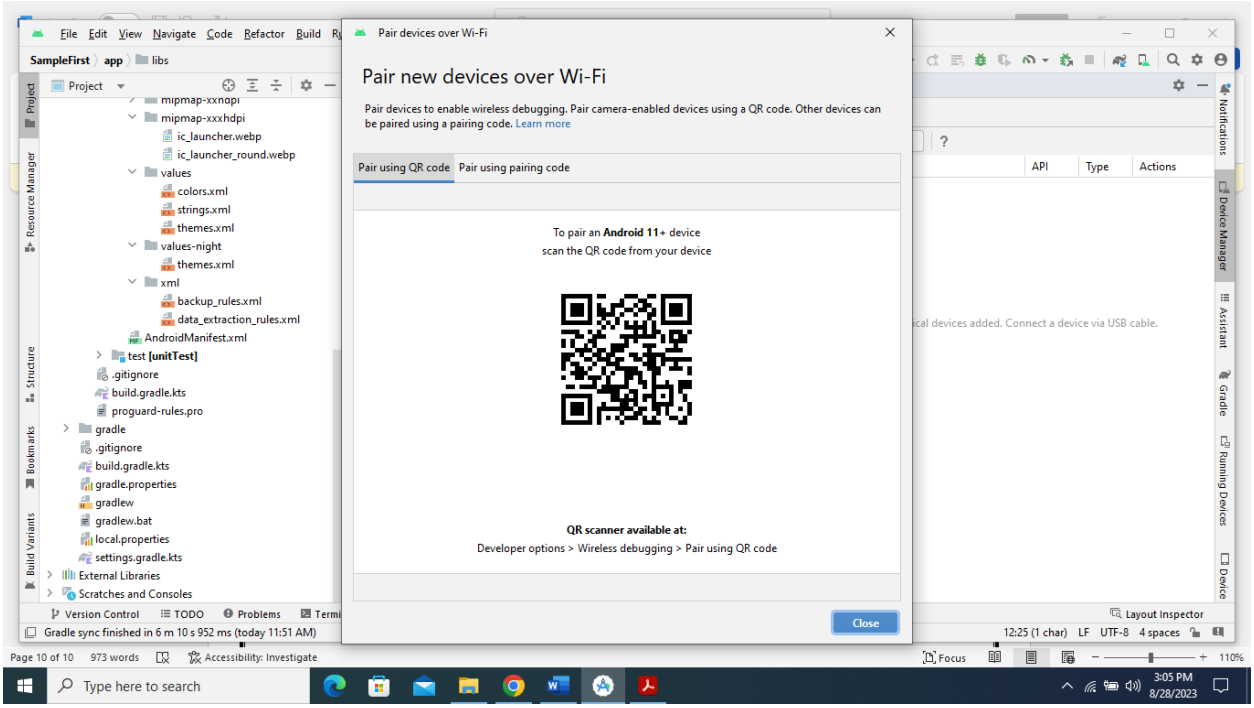
Creation of AVD:

Step 1: Click on Device Manager. By default it contains one virtual emulator with latest API.



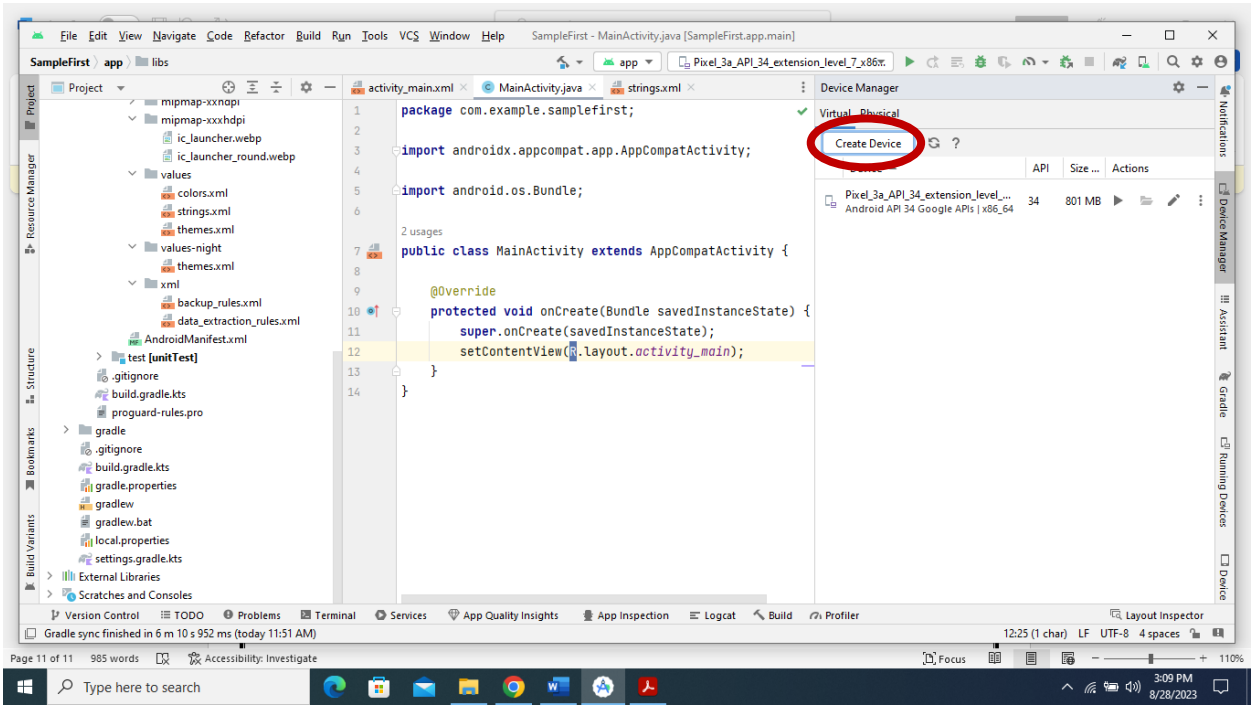
You can run the applications either in virtual device or physical device. If you run the applications using physical device, you need to pair the mobile device with android studio using QR code or using Wi-fi.



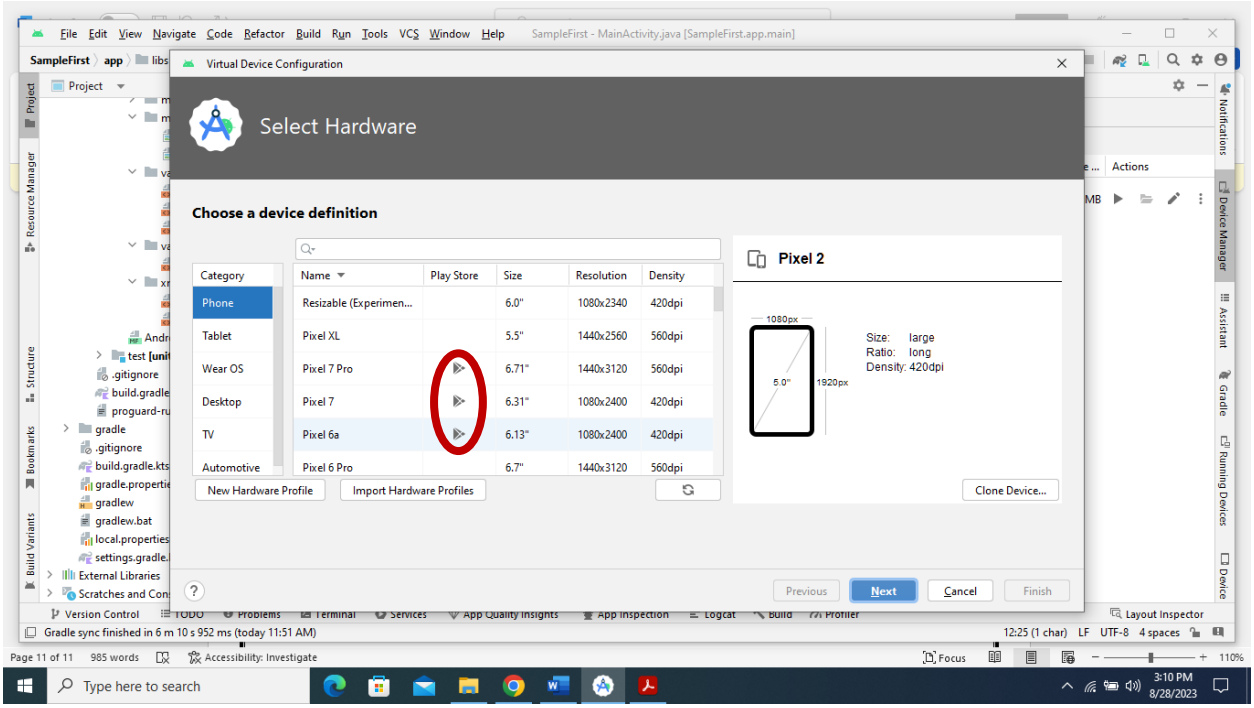


This option is available only android 11+ devices.

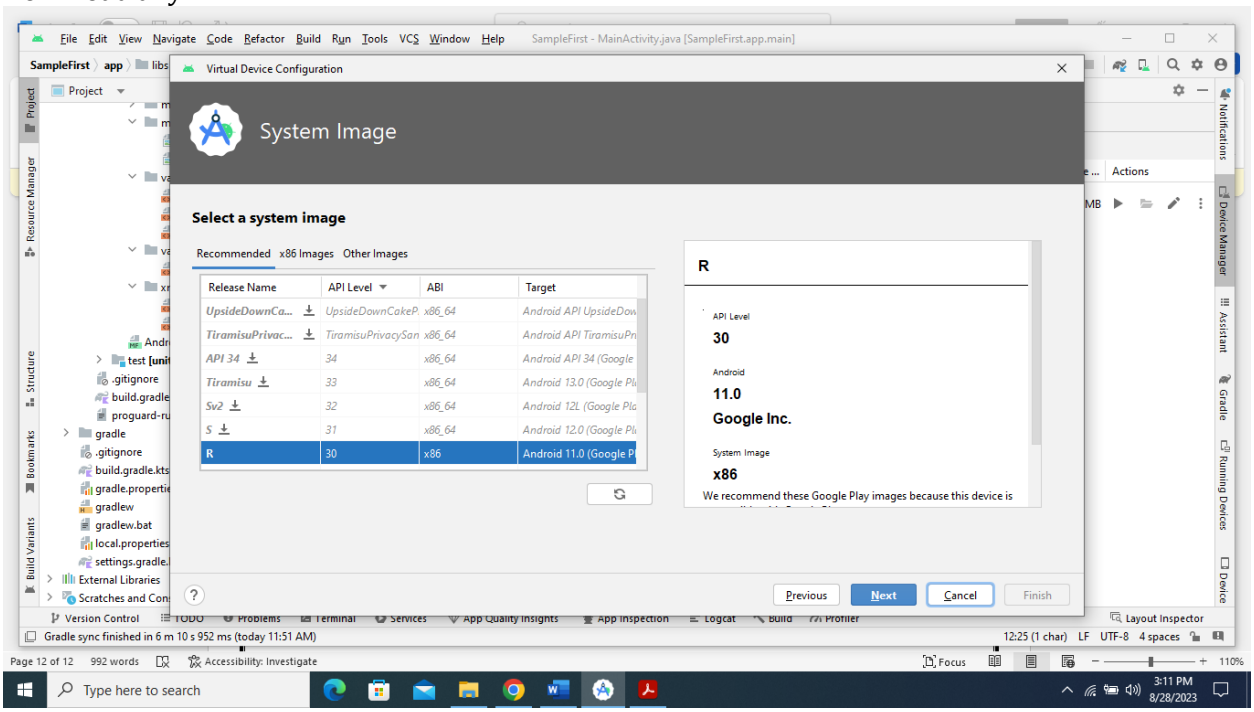
Creation of virtual device:



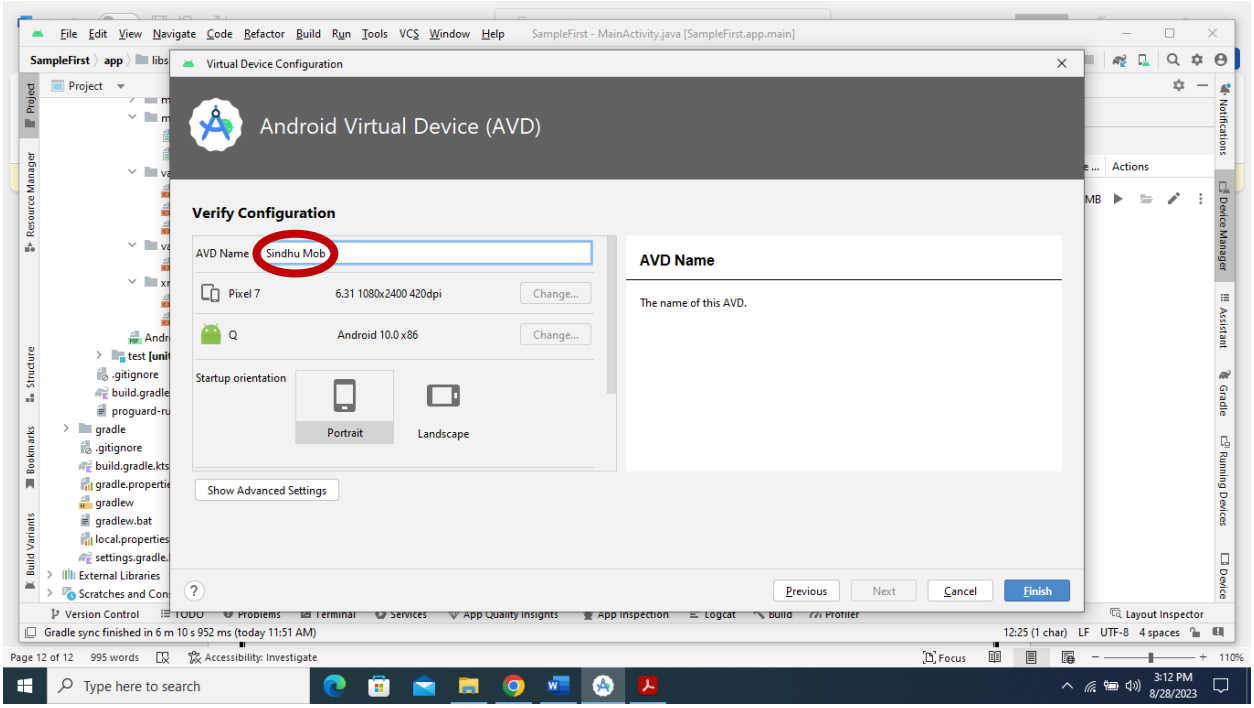
Select any phone, preferably playstore enabled devices



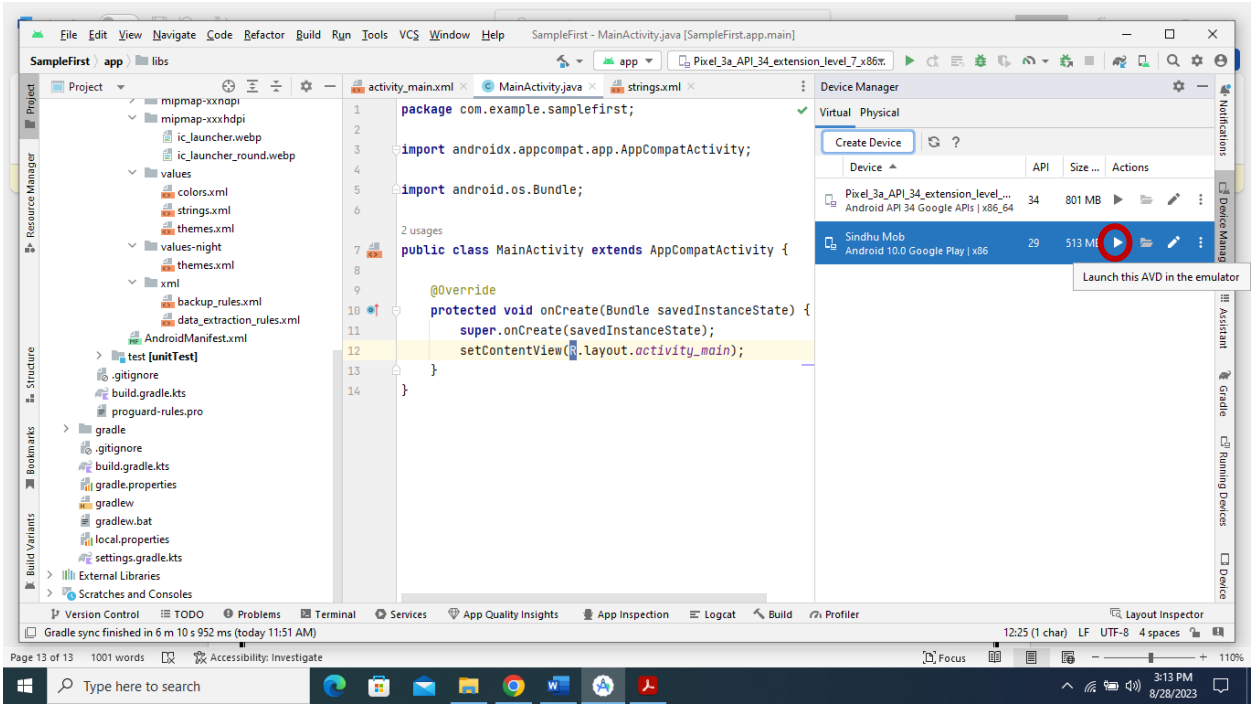
Download any API:



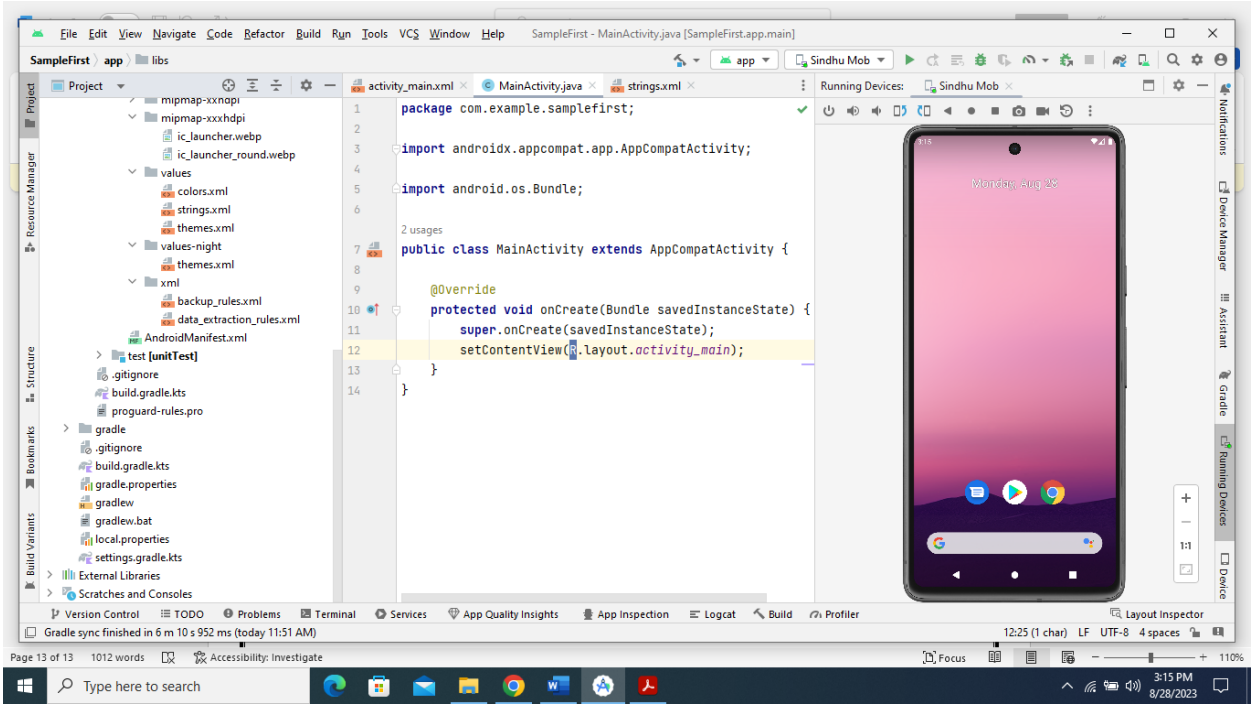
You can change name of AVD:



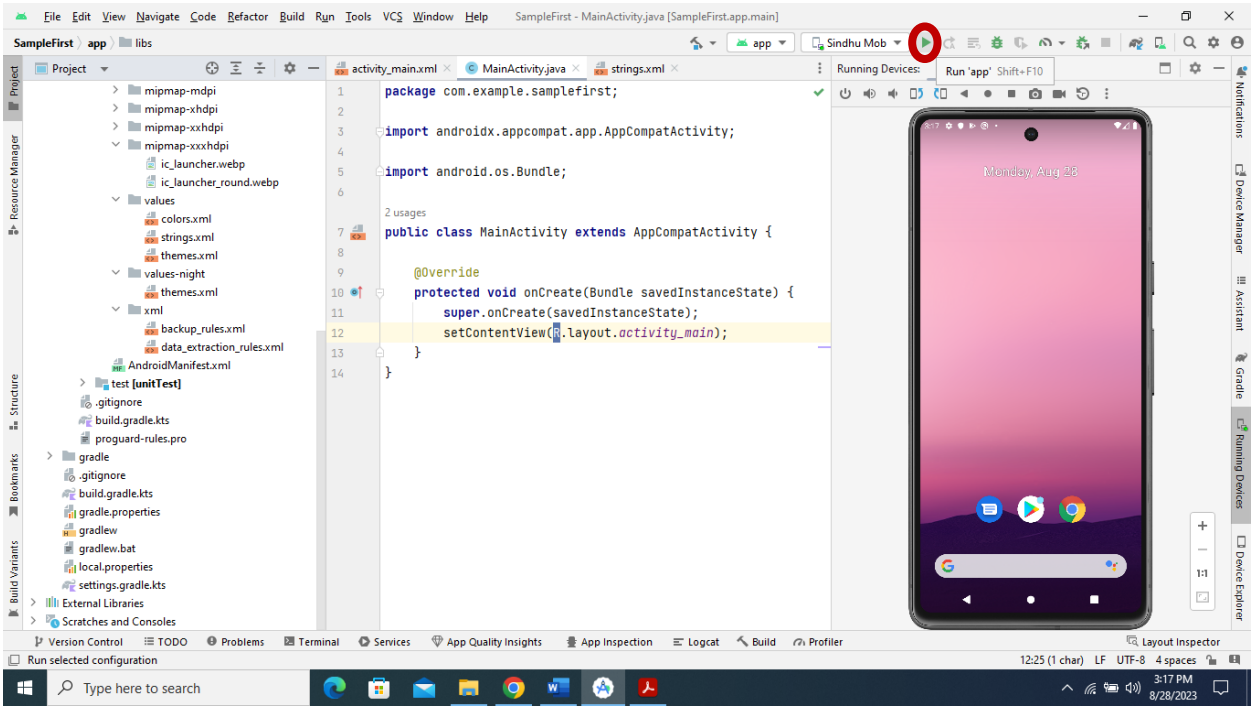
Launch the emulator:



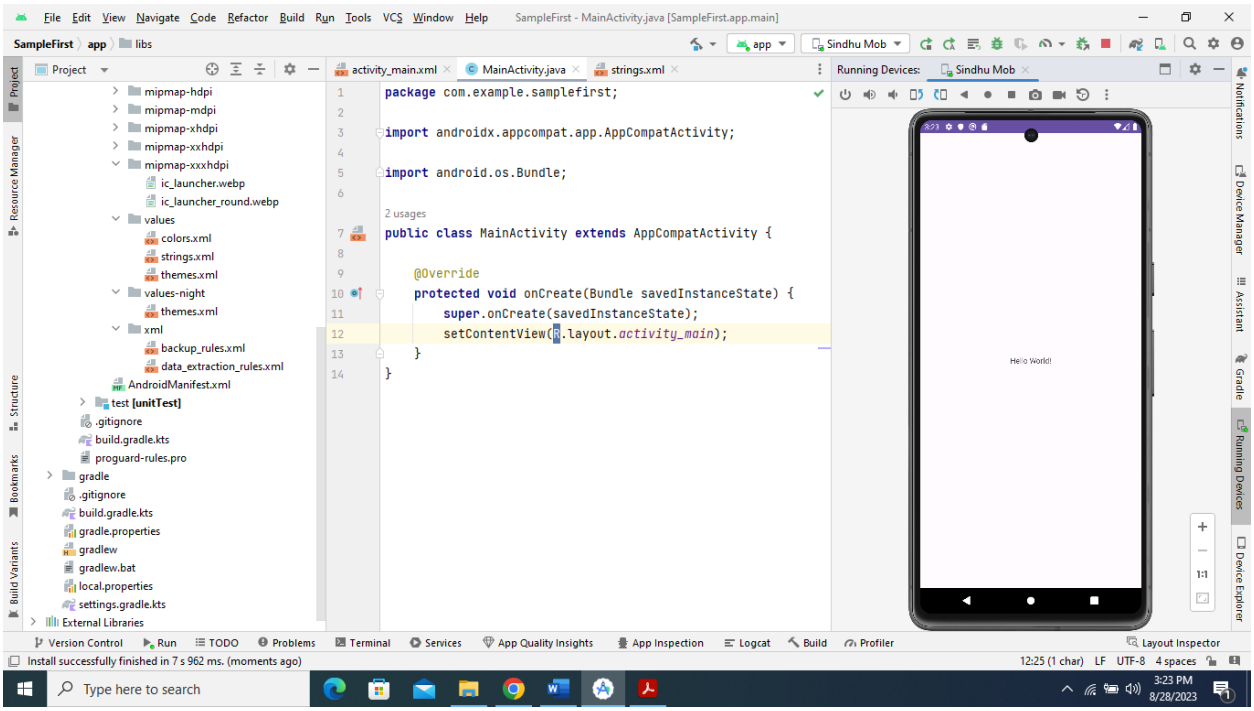
You can see the emulator under running devices:



Running the application:
Let's try to run our Hello World! application we just created. To run the app from android studio click run icon from tool bar



Android studio installs the app on your AVD and starts it and if everything is fine with your setup and application, it will display following Emulator window:


File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
SampleFirst - MainActivity.java [SampleFirst.app.main]
SampleFirst app libs
app
Sindhu Mob
Project
mipmap-hdpi
mipmap-mdpi
mipmap-xhdpi
mipmap-xxhdpi
mipmap-xxxhdpi
ic_launcher.webp
ic_launcher_round.webp
values
colors.xml
strings.xml
themes.xml
values-night
themes.xml
xml
backup_rules.xml
data_extraction_rules.xml
AndroidManifest.xml
test [unitTest]
.gitignore
build.gradle.kts
proguard-rules.pro
gradle
.gitignore
build.gradle.kts
gradle.properties
gradlew
gradlew.bat
local.properties
settings.gradle.kts
External Libraries
activity_main.xml
MainActivity.java
strings.xml
package com.example.samplefirst;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
2 usages
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
}
}
Running Devices:
Sindhu Mob
Hello World!
Version Control
Run
TODO
Problems
Terminal
Services
App Quality Insights
App Inspection
Logcat
Build
Profiler
Layout Inspector
Install successfully finished in 7 s 962 ms. (moments ago)
12:25 (1 char) LF UTF-8 4 spaces
Type here to search
3:23 PM
8/28/2023